

Complete Testing Guide - HyperPay Payment Gateway

Prerequisites

1. Database Setup

```
dotnet ef migrations add CompleteSetup
dotnet ef database update
```

2. Start the Application

```
dotnet run
```

3. Tools Required

- Postman or similar API testing tool
- Web browser for frontend testing

Phase 1: API Testing (Backend Only)

Test 1: Health Check

```
GET https://localhost:44383/health
```

Expected: Status 200, "Healthy"

Test 2: Register Application

```
POST https://localhost:44383/api/apiclient/register
Content-Type: application/json
```

```
{
  "clientName": "Test Ecommerce App"
}
```

Expected Response:

```
{
  "status": 1,
  "message": "Client registered successfully...",
  "data": {
    "clientId": 1,
    "clientName": "Test Ecommerce App",
    "apiKey": "abc123xyz..."
  }
}
```

Action: Copy the `apiKey` - you'll need it for all subsequent tests.

Test 3: Create Checkout Session

```
POST https://localhost:44383/api/payment/checkout
X-API-Key: YOUR_API_KEY_FROM_TEST_2
Content-Type: application/json
```

```
{
  "amount": 100.00,
  "merchantTransactionId": "ORDER-TEST-001",
  "paymentMethod": "CARD",
  "customer": {
    "email": "test@example.com",
    "givenName": "John",
    "surname": "Doe"
  },
  "billing": {
    "street1": "123 Main Street",
    "city": "Riyadh",
    "state": "Riyadh",
    "country": "SA",
    "postcode": "12345"
  }
}
```

Expected Response:

```
{
  "status": 1,
  "message": "Checkout session created successfully",
  "data": {
    "checkoutId": "xxx.yyy.zzz",
    "merchantTransactionId": "ORDER-TEST-001"
  }
}
```

Action: Copy the `checkoutId`.

What This Tests:

- API key authentication
- HyperPay API integration
- Database transaction creation
- Device/IP tracking

Test 4: Check Payment Status (Before Payment)

```
POST https://localhost:44383/api/payment/status
X-API-Key: YOUR_API_KEY
Content-Type: application/json
```

```
{
  "checkoutId": "YOUR_CHECKOUT_ID_FROM_TEST_3"
}
```

Expected: Status will be "PENDING" since no payment was made yet.

Phase 2: End-to-End Payment Testing (With Test Cards)

Step 1: Create Test HTML File

Save as `payment-test.html`:

```
<!DOCTYPE html>
<html>
<head>
```

```
<title>HyperPay Payment Test</title>
<meta charset="UTF-8">
<style>
  body { font-family: Arial; max-width: 800px; margin: 50px auto; padding: 20px; }
  .section { border: 1px solid #ddd; padding: 20px; margin: 20px 0; border-radius: 5px; }
  .test-cards { background: #f0f8ff; padding: 15px; margin: 20px 0; }
  button { background: #007bff; color: white; padding: 10px 20px; border: none; border-radius: 5px; cursor: pointer; margin: 5px; }
  button:hover { background: #0056b3; }
  #result { background: #f8f9fa; padding: 15px; margin: 20px 0; white-space: pre-wrap; font-family: monospace; font-size: 12px; }
</style>
</head>
<body>
  <h1>HyperPay Payment Gateway Test</h1>

  <div class="section">
    <h2>Step 1: Create Checkout</h2>
    <label>API Key:</label><br>
    <input type="text" id="apiKey" style="width: 100%; padding: 8px; margin: 10px 0;" placeholder="Enter your API key">
    <br>
    <label>Amount (SAR):</label><br>
    <input type="number" id="amount" value="100.00" style="width: 200px; padding: 8px; margin: 10px 0;">
    <br>
    <button onclick="createCheckout()">Create Checkout Session</button>
  </div>

  <div class="test-cards">
    <h3>Test Card Details</h3>
    <p><strong>VISA (Success):</strong> 4012000033330026 | Expiry: 01/39 | CVV: 100</p>
    <p><strong>MasterCard (Success):</strong> 5123450000000008 | Expiry: 01/39 | CVV: 100</p>
    <p><strong>Card (Fail):</strong> 5204730000002514 | Expiry: 01/39 | CVV: 251</p>
    <p><strong>MADA (Success):</strong> 4464040000000007 | Expiry: 11/26 | CVV: 850</p>
    <p><em>Cardholder Name: Any Name</em></p>
  </div>

  <div class="section" id="paymentSection" style="display:none;">
    <h2>Step 2: Complete Payment</h2>
    <p>Checkout ID: <span id="checkoutIdDisplay"></span></p>
    <div id="paymentForm"></div>
  </div>

  <div class="section" id="statusSection" style="display:none;">
    <h2>Step 3: Verify Payment</h2>
    <button onclick="checkStatus()">Check Payment Status</button>
  </div>

  <div id="result"></div>

  <script>
    let checkoutId = '';
    let apiKey = '';

    async function createCheckout() {
      apiKey = document.getElementById('apiKey').value;
      const amount = parseFloat(document.getElementById('amount').value);

      if (!apiKey) {
        alert('Please enter API key');
        return;
      }

      const payload = {
        amount: amount,
        merchantTransactionId: 'ORDER-' + Date.now(),
        paymentMethod: 'CARD',
        customer: {
```

```

        email: 'test@example.com',
        givenName: 'John',
        surname: 'Doe'
    },
    billing: {
        street1: '123 Main St',
        city: 'Riyadh',
        state: 'Riyadh',
        country: 'SA',
        postcode: '12345'
    }
};

try {
    const response = await fetch('https://localhost:44383/api/payment/checkout', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'X-API-Key': apiKey
        },
        body: JSON.stringify(payload)
    });

    const result = await response.json();
    document.getElementById('result').textContent = JSON.stringify(result, null, 2);

    if (result.status === 1) {
        checkoutId = result.data.checkoutId;
        document.getElementById('checkoutIdDisplay').textContent = checkoutId;
        loadPaymentWidget();
        document.getElementById('paymentSection').style.display = 'block';
        document.getElementById('statusSection').style.display = 'block';
    }
} catch (error) {
    document.getElementById('result').textContent = 'Error: ' + error.message;
}

function loadPaymentWidget() {
    const script = document.createElement('script');
    script.src = `https://eu-test.oppwa.com/v1/paymentWidgets.js?checkoutId=${checkoutId}`;
    document.head.appendChild(script);

    script.onload = () => {
        document.getElementById('paymentForm').innerHTML = `
            <form action="payment-result.html?checkoutId=${checkoutId}&apiKey=${apiKey}" class="paymentWidgets" data-brands=
            `;
    };
};

async function checkStatus() {
    try {
        const response = await fetch('https://localhost:44383/api/payment/status', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'X-API-Key': apiKey
            },
            body: JSON.stringify({ checkoutId: checkoutId })
        });

        const result = await response.json();
        document.getElementById('result').textContent = JSON.stringify(result, null, 2);
    } catch (error) {
        document.getElementById('result').textContent = 'Error: ' + error.message;
    }
}

```

```

    }
  }
</script>
</body>
</html>

```

Step 2: Create Result Page

Save as `payment-result.html` :

```

<!DOCTYPE html>
<html>
<head>
  <title>Payment Result</title>
  <style>
    body { font-family: Arial; max-width: 600px; margin: 50px auto; padding: 20px; }
    .success { background: #d4edda; padding: 20px; border-radius: 5px; color: #155724; }
    .failed { background: #f8d7da; padding: 20px; border-radius: 5px; color: #721c24; }
    button { background: #007bff; color: white; padding: 10px 20px; border: none; border-radius: 5px; cursor: pointer; margin: 10px; }
    #details { background: #f8f9fa; padding: 15px; margin: 20px 0; white-space: pre-wrap; font-family: monospace; font-size: 12px; }
  </style>
</head>
<body>
  <h1>Payment Result</h1>
  <div id="status">Checking payment status...</div>
  <button onclick="checkAgain()">Refresh Status</button>
  <button onclick="window.location.href='payment-test.html'">New Payment</button>
  <div id="details"></div>

  <script>
    const urlParams = new URLSearchParams(window.location.search);
    const checkoutId = urlParams.get('checkoutId');
    const apiKey = urlParams.get('apiKey');

    async function checkStatus() {
      try {
        const response = await fetch('https://localhost:44383/api/payment/status', {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json',
            'X-API-Key': apiKey
          },
          body: JSON.stringify({ checkoutId: checkoutId })
        });

        const result = await response.json();
        document.getElementById('details').textContent = JSON.stringify(result, null, 2);

        if (result.status === 1) {
          const paymentStatus = result.data.paymentStatus;
          if (paymentStatus === 'SUCCESS') {
            document.getElementById('status').className = 'success';
            document.getElementById('status').innerHTML = `
              <h2>✓ Payment Successful!</h2>
              <p>Transaction ID: ${result.data.transactionId}</p>
              <p>Amount: ${result.data.amount} ${result.data.currency}</p>
            `;
          } else {
            document.getElementById('status').className = 'failed';
            document.getElementById('status').innerHTML = `
              <h2>✗ Payment Failed</h2>
              <p>${result.data.resultDescription}</p>
            `;
          }
        }
      } catch {
        // Handle error
      }
    }

    checkStatus();
  </script>

```

```

    } catch (error) {
        document.getElementById('details').textContent = 'Error: ' + error.message;
    }
}

function checkAgain() {
    checkStatus();
}

// Auto-check on load
checkStatus();
</script>
</body>
</html>

```

Step 3: Run Complete Test

1. Open `payment-test.html` in browser
2. Enter your API key
3. Click "Create Checkout Session"
4. Enter test card details (use 4012000033330026 for success)
5. Complete payment
6. Verify status shows SUCCESS

Phase 3: Security Testing

Test 5: Rate Limiting

Make 65 requests within 1 minute to the same endpoint.

Expected: After 60 requests, you should get:

```

{
  "status": 0,
  "message": "Rate limit exceeded. Please try again later.",
  "data": null
}

```

Test 6: Invalid API Key

```

POST https://localhost:44383/api/payment/checkout
X-API-Key: invalid-key

```

Expected: 401 Unauthorized

Test 7: Missing Required Fields

```

POST https://localhost:44383/api/payment/checkout
X-API-Key: YOUR_API_KEY
Content-Type: application/json

{
  "amount": 100.00
}

```

Expected:

```
{
  "status": 0,
  "message": "MerchantTransactionId is required",
  "data": null
}
```

Phase 4: Admin Testing

Test 8: Global Statistics

```
GET https://localhost:44383/api/admin/stats/global
X-Admin-Key: admin_key_123@!#$
```

Test 9: View All Transactions

```
GET https://localhost:44383/api/admin/transactions/all?page=1&pageSize=10
X-Admin-Key: admin_key_123@!#$
```

Test 10: Stats by Application

```
GET https://localhost:44383/api/admin/stats/by-application
X-Admin-Key: admin_key_123@!#$
```

Test Card Scenarios

Card Number	Type	Expiry	CVV	Expected Result
4012000033330026	VISA	01/39	100	SUCCESS
5123450000000008	MasterCard	01/39	100	SUCCESS
5204730000002514	Card	01/39	251	FAILED
4464040000000007	MADA	11/26	850	SUCCESS

Important: Use whole amounts (100.00, 50.00) for successful test transactions.

Verification Checklist

After completing all tests, verify:

- ☐ Checkout sessions are created successfully
 - ☐ Transactions are saved in database with PENDING status
 - ☐ Payment widget loads correctly
 - ☐ Test cards process successfully
 - ☐ Payment status updates to SUCCESS/FAILED
 - ☐ Card details (last 4 digits, brand) are captured
 - ☐ Device information is recorded
 - ☐ Rate limiting works
 - ☐ API key authentication works
 - ☐ Admin endpoints require admin key
 - ☐ Each application sees only their transactions
 - ☐ Health check endpoint responds
-

Troubleshooting

Issue: "Invalid or inactive API Key"

- **Solution:** Ensure you're using the correct API key from registration

Issue: Payment status stays PENDING

- **Solution:** You must complete payment through the widget with test cards

Issue: "Rate limit exceeded"

- **Solution:** Wait 1 minute or restart the application

Issue: Database connection error

- **Solution:** Check PostgreSQL is running and connection string is correct
-

Production Readiness Checklist

Before deploying to production:

- ☐ Change `TestMode` to `false` in `appsettings.json`
 - ☐ Update to production HyperPay credentials
 - ☐ Change `AdminKey` to a secure random value
 - ☐ Store sensitive config in environment variables/Azure Key Vault
 - ☐ Update `AllowedOrigins` with production domains
 - ☐ Enable HTTPS only
 - ☐ Set up monitoring and logging
 - ☐ Configure database backups
 - ☐ Test with production cards (small amounts)
 - ☐ Set up webhook for real-time payment notifications
-

Next Steps

1. Integrate with your e-commerce frontend
2. Implement webhook handler for real-time updates
3. Add refund functionality
4. Set up monitoring and alerts
5. Configure automated backups